

字符的世界

字符串处理 · 12题 · C++ & Python 双语对照

小鲁已经学会了数字的处理——整数、浮点数、数组。但真实世界的程序处理得最多的不是数字，而是文字。从搜索引擎到聊天软件，从代码编辑器到语音助手——字符串处理是每个程序员的基本功。而Python在字符串处理上的优势几乎是碾压级别的：切片、join、split、replace.....C++需要好几行的操作，Python往往一行解决。

本章12道题，涵盖了字符串六大核心操作：长度与遍历、字符分类、修改与替换、切片与拼接、频次统计、分词与重构。学完之后你会发现——用Python处理字符串是编程中最快乐的事之一。

本章角色

-  小鲁（初学者）· 从"数字世界"迈入"文字世界"，发现Python字符串处理的魔法
-  小华（算法高手）· 擅长对比C++和Python的字符串处理差异，强调"理解原理"
-  小栋（工程师）· 用"文本编辑器""搜索引擎"等实际场景出题
-  小嘉（校主精神）· 偶尔提及密码学和南洋通信的故事，赋予字符串处理历史深度

Python字符串处理四大法宝

- 切片：s[a:b]截取[a,b)区间。s[:i]前缀、s[i:]后缀、s[::-1]反转
- 分割与拼接：s.split(sep)按分隔符拆分，sep.join(list)按分隔符拼接
- 查找与替换：s.find(sub)、s.replace(old,new)、sub in s 成员判断
- 字符分类：isdigit()/isalpha()/isupper()/islower()/isspace() 自动分类
- 频次统计：collections.Counter(s) 一行建字典。字符→频次的O(1)映射

C++注意：字符串分为std::string（类）和char[]（C风格），后者需要strlen/strcpy/strcat。现代C++推荐用std::string。

NQ067 字符串长度 AcWing 760

小鲁输入了一串文字，想知道它有多长。"这还用编程？数一下不就行了....."话没说完，小华就打断了他："如果有人给你一本1000页的书，你一个字一个字数吗？Python用len(s)一秒搞定。C++用s.length()或strlen()。字符串长度是所有文本处理的基础——搜索、替换、截取，第一步永远是知道字符串有多长。"

输入 一行，一个字符串（不超过100字符）。

输出 一个整数，表示字符串长度。

范围 字符串长度≤100

输入

hello world

输出

11

思路 字符串长度是最基本的信息。Python: len(s), C++: s.length()或s.size() (string类) 或strlen(s) (C风格char数组)。注意空格也算长度——"hello world"是11个字符不是10个。

技巧 Python的len()适用于所有序列类型 (字符串/列表/元组), 是一个全局函数而非方法。C++的length()和size()对于string完全等价。C风格字符串用strlen()但需要#include。

C++

```
#include <cstdio>

int main()
{
    char str[101];

    fgets(str, 101, stdin);

    int len = 0;
    for (int i = 0; str[i] && str[i] != '\n'; i++) len++;

    printf("%d\n", len);

    return 0;
}
```

Python

```
# Python solution
```

NQ068 字符串中的数字个数 AcWing 761

"帮我统计这篇文章里有多少个数字字符....."小栋拿着一份报纸对小鲁说。小鲁刚要用手指一个一个点, 小华按住他: "遍历字符串的每个字符, 用isdigit()判断是不是数字。Python一行搞定——sum(c.isdigit() for c in s)。C++则是for循环判断c>='0' && c<='9'。"小鲁这才意识到字符串里的每个字符都可以被'分类'——数字、字母、空格、标点.....每种分类都有对应的判断方法。

输入 一行, 一个字符串 (不超过100字符)。

输出 字符串中数字字符的个数。

范围 字符串长度≤100

输入

I am 20 years old.

输出

2

思路 分类统计模式: 遍历每个字符, 判断是否为数字 ('0'-'9')。Python: sum(c.isdigit() for c in s), C++: 循环+条件判断。isdigit()是Python字符串方法的强大之处。

技巧 Python字符方法: s.isdigit() (数字)、s.isalpha() (字母)、s.isalnum() (字母数字)、s.isspace() (空白)。记住这些方法能让字符串处理效率翻倍。

C++

```
#include <cstdio>

int main()
{
    char str[101];

    fgets(str, 101, stdin);

    int cnt = 0;
    for (int i = 0; str[i]; i++)
        if (str[i] >= '0' && str[i] <= '9')
            cnt++;

    printf("%d\n", cnt);

    return 0;
}
```

Python

```
s = input()
print(sum(1 for c in s if c.isdigit()))
```

NQ069 循环相克令 AcWing 763

小鲁和小栋在玩"锤子剪刀布"的游戏——不过他们的版本叫"Hunter/Bear/Gun"。"猎人怕熊，熊怕枪，枪怕猎人——形成一个循环克制关系。"小华说，"这种循环相克的问题可以用字符串比较解决。但更优雅的是数学方法：把每个角色映射成0/1/2，然后胜者 $=(a-b+3)\%3$ 。"小鲁惊奇地发现——一个简单的取模竟然能判断所有博弈结果。

输入 一个整数N。然后N行每行两个字符串（Hunter/Bear/Gun）。

输出 对每行输出Player1赢还是Player2赢。

范围 $1 \leq N \leq 100$

输入	输出
3	Player2
Hunter Bear	Player1
Bear Hunter	Player1
Gun Bear	

思路 字符串比较法：if-elif分支判断所有9种组合。数学法：将三个角色映射为0,1,2，则P1赢当且仅当 $(a-b+3)\%3==2$ 。数学法将9个if压缩成一行——用数学替代分支的艺术。

技巧 Python用法：dic={'Hunter':0,'Bear':1,'Gun':2}，然后if (dic[a]-dic[b]+3)%3==2: P1赢。C++用map或直接if-else。理解取模在循环问题中的通用性更关键。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    int n;
    cin >> n;

    while (n -- )
    {
        string a, b;
        cin >> a >> b;

        int x, y;
        if (a == "Hunter") x = 0;
        else if (a == "Bear") x = 1;
        else x = 2;

        if (b == "Hunter") y = 0;
        else if (b == "Bear") y = 1;
        else y = 2;

        if (x == y) puts("Tie");
        else if (x == (y + 1) % 3) puts("Player1");
        else puts("Player2");
    }

    return 0;
}
```

Python

```
a, b = input().split()
w = {("Hunter", "Gun"), ("Bear", "Hunter"), ("Gun", "Bear")}
if a == b:
    print("Tie")
elif (a, b) in w:
    print("Player1")
else:
    print("Player2")
```

NQ070 字符串加空格 AcWing 765

小鲁在排版一个标题——需要给每个字符之间加上空格。"这要手动打多少个空格……"小华笑了："Python的join()方法就是为此而生的：' '.join(s)——用空格把字符串的每个字符连接起来。C++需要遍历输出每个字符后面加空格然后去掉末尾空格。两种语言对比，Python的join()堪称字符串处理的魔法。"

输入 一行，一个字符串（不含空格）。

输出 每个字符间加空格后的字符串。

范围 字符串长度≤100

输入

hello

输出

h e l l o

思路 join()方法将一个可迭代对象用分隔符连接。' '.join(s)把s的每个字符用空格连接。C++版需要for循环逐个输出注意首尾空格。Python的join()是最常用的字符串操作之一。

技巧 Python的join()和split()是互逆操作。'sep'.join(list)把列表用sep连起来，str.split(sep)把字符串按sep拆开。这两个方法是字符串处理的基础中的基础。

C++

```
#include <iostream>

using namespace std;

int main()
{
    string a;
    getline(cin, a);

    string b;
    for (auto c : a) b = b + c + ' ';

    b.pop_back();

    cout << b << endl;

    return 0;
}
```

Python

```
print(" ".join(input()))
```

NQ071 替换字符 AcWing 769

小鲁在编辑文本时发现有些字符需要换成别的："比如把所有的'a'换成'b'....."小华说："Python直接s.replace(old, new)一行搞定。C++要手动遍历替换。不过注意——Python的字符串是不可变的，replace()返回的是新字符串，原字符串不变。理解'不可变性'是理解Python字符串操作的关键。"

输入

一行字符串（可能含空格）。然后一行，两个字符（要替换的和替换成的）。

输出

替换后的字符串。

范围

字符串长度≤100

输入

```
hello world
o e
```

输出

```
helle world
```

思路 字符替换：遍历每个字符，如果等于目标则输出新字符，否则原样输出。Python: s.replace(old, new)。C++: 用for+if或std::replace()。注意Python的字符串不可变性——replace()返回新的。

技巧 Python字符串不可变意味着所有'修改'操作都返回新字符串。这保证了字符串的安全性（不会被意外修改），但创建大量新字符串时有性能开销。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char str[31];
    scanf("%s", str);

    char c;
    scanf("\n%c", &c);

    for (int i = 0; str[i]; i++)
        if (str[i] == c)
            str[i] = '#';

    puts(str);

    return 0;
}
```

Python

```
s = input()
c = input()
print(s.replace(c, "#"))
```

NQ072 字符串插入 AcWing 773

"如果我想在字符串的第3个位置插入另一个字符串怎么办?"小鲁问。小华说:"拿到前面部分、后面部分,然后拼接: `s[:p]+sub+s[p:]`。注意位置`p`是从0开始还是从1开始——这是字符串操作中最容易出错的off-by-one问题。"小鲁立刻想到:"所以插入的本质就是'切片+拼接'——把字符串切成两段,中间塞进新内容。"

输入 三行: 原字符串、要插入的字符串、插入位置。

输出 插入后的字符串。

范围 字符串长度 ≤ 100 , 位置从0开始

输入

```
hello
world
3
```

输出

```
helworldlo
```

思路 Python切片拼接: `s[:p]+sub+s[p:]`。C++: `substr()`取前后两部分再相加。切片是Python最优雅的特性之一——`s[a:b]`精确截取`[a,b)`区间, 语义清晰。

技巧 Python切片`s[a:b]`的区间是`[a,b)` (包含`a`不包含`b`)。插入位置`p`意味着新内容从`p`开始——`s[:p]`取`p`之前的字符, `s[p:]`取`p`及之后的字符。

C++

```
#include <iostream>
#include <cstring>
using namespace std;
int main()
{
    string a,b;
    while(cin>>a>>b){

        int p=0;
        for (int i=0;i<a.size();i++)
            if (a[i]>a[p]) p=i;//找到最大的位置

        cout<<a.substr(0,p+1)+b+a.substr(p+1)<<endl;

    }
    return 0;
}
```

Python

```
s = input()
t = input()
i = len(s) // 2
print(s[:i] + t + s[i:])
```

NQ073 只出现一次的字符 AcWing 772

"我要在一段文字中找到第一个只出现一次的字符。"小栋又在给小鲁出题。"像'hello'里——'h','e','o'都只出现一次，但第一个是'h'。"小华说："这需要两遍遍历：第一遍统计每个字符的出现次数，第二遍按顺序找出第一个次数为1的字符。Python用collections.Counter可以一行建好频次字典。"

输入

一个字符串。

输出

第一个只出现一次的字符的位置（1-indexed），如果没有则输出-1。

范围

字符串长度 $\leq 10^5$

输入

hello

输出

1

思路 两遍遍历法：第一遍建立字符频次字典（Counter），第二遍按原字符串顺序查找第一个频次为1的字符。Python用Counter一键统计。注意频次统计不改变顺序——要按原串顺序输出。

技巧 Python: from collections import Counter; cnt=Counter(s); for i,c in enumerate(s): if cnt[c]==1: print(i+1)。C++用int cnt[256]数组直接作哈希表——字符ASCII值作为索引，O(1)频次查找。

C++

```
#include <iostream>
#include <cstring>

using namespace std;

int cnt[26];
char str[100010];

int main()
{
    cin >> str;

    for (int i = 0; str[i]; i++) cnt[str[i] - 'a']++;

    for (int i = 0; str[i]; i++)
        if (cnt[str[i] - 'a'] == 1)
        {
            cout << str[i] << endl;
            return 0;
        }

    puts("no");

    return 0;
}
```

Python

```
s = input()
from collections import Counter
cnt = Counter(s)
for c in s:
    if cnt[c] == 1:
        print(c)
        break
else:
    print("no")
```

NQ074 字符串匹配 AcWing 762

"在一大段文字里，找一小段模式——这就是字符串匹配。"小华在白板上写着，"比如在'abababa'里找'aba'。最朴素的方法：从每个位置开始，逐个字符比较——这叫暴力匹配， $O(n \times m)$ 。后续你会学到更高效的KMP算法，但暴力匹配首先让你理解匹配的本质：滑动窗口+逐位比较。"

输入 两行：文本串T和模式串P。

输出 P在T中第一次出现的位置（1-indexed），若不存在输出-1。

范围 $1 \leq |T|, |P| \leq 100$

输入

abababa
aba

输出

1

思路 暴力匹配：对于i从0到n-m，检查T从i开始的m个字符是否等于P。Python直接用T.find(P)一行，但手写暴力匹配理解原理更重要。find()返回首次出现位置（-1表示不存在）。

技巧 Python的T.find(P)底层是高效的混合算法（Boyer-Moore-Horspool），比手写暴力快得多。但手写暴力匹配是KMP和BM算法的基础——理解"为什么慢"才能理解"如何变快"。

C++

```
#include <iostream>

using namespace std;

int main()
{
    double k;
    string a, b;
    cin >> k >> a >> b;

    int cnt = 0;
    for (int i = 0; i < a.size(); i++)
        if (a[i] == b[i])
            cnt++;

    if ((double)cnt / a.size() >= k) puts("yes");
    else puts("no");

    return 0;
}
```

Python

```
s = input()
t = input()
print("yes" if t in s else "no")
```

NQ075 信息加密 AcWing 767

小鲁对密码学产生了兴趣："如果能写个程序把每个字母往后移1位——a变b，b变c……z回到a——那就成了一个最简单的加密程序！"小华说："这就是凯撒密码，两千年前的加密方法。编程关键是处理循环——'z'+1要变回'a'。可以用取模： $(c - 'a' + 1) \% 26 + 'a'$ 。大写和小写分别处理。"

输入 一个字符串（只含字母和空格）。

输出 字母后移1位，空格不变。

范围 字符串长度 ≤ 100

输入

abc XYZ

输出

bcd YZA

思路 字符偏移+循环回绕。对字母： $c = 'a' + (c - 'a' + 1) \% 26$ 。大写字母同理用'A'和'Z'。非字母（空格等）原样保留。用isalpha()判断是否为字母，用isupper()/islower()判断大小写。

技巧 Python: $\text{chr}((\text{ord}(c) - \text{base} + 1) \% 26 + \text{base})$ 处理循环偏移。C++直接做字符运算。理解ASCII码编码规则——字母是连续排列的，这是字符运算的前提。

C++

```
#include <iostream>
#include <cstring>
using namespace std;

int main(){

    string s;
    getline(cin,s);//读入一行

    for (auto &c:s)//使用auto注意&c直接引用字符串直接修改
        if (c>='a' && c<='z')
            c = 'a' + (c-'a'+1) % 26; //取模运算
        else if (c>='A' && c<='Z')
            c = 'A' + (c-'A'+1) % 26;

    cout<<s<<endl;//输出结果
}
```

Python

```
r = []
for c in input():
    if "a" <= c <= "z":
        r.append(chr((ord(c) - 97 +
1) % 26 + 97))
    elif "A" <= c <= "Z":
        r.append(chr((ord(c) - 65 +
1) % 26 + 65))
    else:
        r.append(c)
print(''.join(r))
```

NQ076 输出字符串 AcWing 764

"这道题有点怪——输入两个字符串A和B，对于B的每个位置，如果它不是A对应位置的字符，就把这个位置到B末尾的子串输出来。"小鲁读了三遍题目才明白。小华解释道："逐位比较A和B。一旦发现某个位置i的字符不同，就输出B从i开始的后缀子串。如果完全相同就输出A。关键是理解'后缀'概念——字符串从某位置到末尾的部分。"

输入 两行：字符串A和B。

输出 按规则输出。

范围 $1 \leq |A|, |B| \leq 100$

输入

abcdef
abcxyz

输出

xyz

思路 逐位比较：if $A[i] \neq B[i]$: 输出 $B[i:]$ 并结束。Python的 $B[i:]$ 切片取后缀。这道题训练的是字符串逐位比较+后缀切片的概念。

技巧 Python切片取后缀： $s[i:]$ 表示从i到末尾。 $s[:i]$ 表示从开头到i-1。切片的三个参数[start:end:step]可以省略任意部分——省略start表示从0开始，省略end表示到末尾。

C++

```
#include <iostream>

using namespace std;

int main()
{
    string a, b;
    getline(cin, a);

    for (int i = 0; i < a.size(); i ++ ) b += (char)(a[i]
+ a[(i + 1) % a.size()]);

    cout << b << endl;

    return 0;
}
```

Python

```
s = input()
print(''.join(chr(ord(c) + 1) for c in s))
```

NQ077 单词替换 AcWing 770

小鲁在写文本编辑器的一个功能：把所有出现的某个单词替换成另一个单词。"比如把新闻里所有的'北京'改成'Beijing'....."小华摇摇头："这个比你想象的复杂！如果用简单的replace，'北京大学'就会变成'Beijing大学'。在OJ环境下，输入是以空格分隔的单词——所以用split()分割后逐词判断替换，再用join()拼回去，更安全也更清晰。"

输入 三行：原文句子、要替换的单词、替换成的单词。

输出 替换后的句子。

范围 字符串长度≤1000

输入

I like apple
apple
orange

输出

I like orange

思路 分词→逐词判断→重建句子。Python: words=s.split(); result=' '.join(w if w!=old else new for w in words)。注意区分单词和子串——用split和join保证按词替换而非子串替换。

技巧 Python的split()默认按任意空白字符（空格、Tab、换行）分割。' '.join()则按单空格重新拼接。这两个方法组合是Python文本处理的黄金搭档。

C++

```
#include <iostream>
#include <sstream>

using namespace std;

int main()
{
    string s, a, b;

    getline(cin, s);
    cin >> a >> b;

    stringstream ssin(s);
    string str;
    while (ssin >> str)
        if (str == a) cout << b << ' ';
        else cout << str << ' ';

    return 0;
}
```

Python

```
s=input();a=input();b=input()
print(' '.join(b if w==a else w for w
in s.split()))
```

NQ078 最长单词 AcWing 774

"在一句话里找最长的单词——小学生也能做。"小鲁不屑地说。"那如果有两个一样长的呢？输出先出现的那个。"小栋又把条件加复杂了。小华说："用split()分词，然后遍历找最长的。Python一行就能搞定——max(words, key=len)。注意要去掉末尾的句号（这是一个小陷阱）。"

输入

一个英文句子，以'.'结尾。

输出

最长单词。若有多个，输出最先出现的。

范围

句子长度≤500

输入

I love programming.

输出

programming

思路 去掉句号→split()分词→max()找最长。Python: s.rstrip('.').split()得到单词列表，max(words, key=len)选出最长。注意max()默认返回最先出现的（稳定性）。

技巧 Python的max()在key相同时保持原始顺序（稳定排序）。rstrip('.')去掉末尾指定字符——这是去除标点的小技巧。split()不加参数会去掉所有空白字符。

C++

```
#include <iostream>

using namespace std;

int main()
{
    string res, str;

    while (cin >> str)
    {
        if (str.back() == '.') str.pop_back();
        if (str.size() > res.size()) res = str;
    }

    cout << res << endl;

    return 0;
}
```

Python

```
words=input().split()
print(max(words,key=len))
```

本章知识点总结

- 字符串六大操作：长度遍历、字符分类、修改替换、切片拼接、频次统计、分词重构
- Python四大法宝：切片s[a:b]、split+join、find+replace、isdigit+isalpha——一行代码等于C++五行的力量
- 字符串不可变：Python字符串修改操作全部返回新对象。理解这点才能正确使用replace/upper/lower等
- 暴力匹配：理解O(n×m)的朴素算法才能理解KMP的O(n+m)有多快。先学走再学跑
- 编码知识：ASCII码中字母连续排列，这是字符偏移加密的数学基础
- 厦大精神：自强不息，止于至善。从整数到字符串，你在成为更全面的程序员